Prompt Engineering Discipline: The Prompt Ladder

Baseline Run

The Prompt:

> *"Write a form for user settings in React."*

Output Excerpt:

> "```jsx
> const SettingsForm = () => {
> const [name, setName] = useState("");
> return (<form><input type="text" value={name} onChange={e => setName(e.target.value)}
> /><button>Submit</button></form>)
> }
> ```
> *Make sure to style this using standard CSS or inline styles as preferred.*"

Version 1: Layer Added — Clearer Goal

The Prompt:

> *"Write a client-side settings form component for a Next.js App Router application that handles user name and phone updates."*

Output Excerpt:

> "```tsx
> // app/settings/page.tsx
> "use client";
> import { useState } from "react";
> export default function SettingsForm() {
> const [phone, setPhone] = useState("");
> // Basic HTML form layout using inline states...
> }
> ```"

1. What Changed:	Switched from generic 'React' to a precise 'Next.js App Router client-side component targeting explicit input fields'.
2. What Improved:	The output dropped generic boilerplate and generated a valid modern Next.js 'use client' file path structure.
3. What Failed:	The internal state structure was still primitive raw HTML state hooks, completely missing validation or loading behaviors.

4. Next Experiment:	Introduce modern architectural context by bringing in structural library dependencies like react-hook-form and Zod.

Version 2: Layer Added — Real Context (Dependencies)

The Prompt:

> *"Write a client-side settings form component for a Next.js App Router application that handles user name and phone updates. You must use react-hook-form paired with zod for schema validation."*

Output Excerpt:

> "```tsx
> import { useForm } from "react-hook-form";
> import { zodResolver } from "@hookform/resolvers/zod";
> import * as z from "zod";
> const schema = z.object({ phone: z.string() });
> // Form layout initializing schema resolution...
> ```"

1. What Changed:	Enforced explicit third-party libraries (`react-hook-form` + `zod`) to target modern production workflows.
2. What Improved:	The output dropped primitive unmanaged standard states entirely and correctly structured a form validation schema wrapper.
3. What Failed:	The schema validation logic was completely bare—accepting any arbitrary string for a telephone input without structural validation rules.
4. Next Experiment:	Impose absolute data validation criteria constraints to handle production inputs safely.

Version 3: Layer Added — Constraints (Strict Validation)

The Prompt:

> *"Write a client-side settings form component for a Next.js App Router application that handles user name and phone updates. You must use react-hook-form paired with zod for schema validation. Constraints: Phone validation must strictly enforce an international E.164 format via a regex pattern."*

Output Excerpt:

> "```tsx
> const schema = z.object({
> name: z.string().min(2),
> phone: z.string().regex(/^\+[1-9]\d{1,14}$/, "Invalid E.164 phone number")
> });
> ```"

1. What Changed:	Added explicit validation formatting rules using the E.164 standardization constraint criteria.
2. What Improved:	The schema evolved from a basic string format to a highly precise international regular expression logic match.
3. What Failed:	The AI over-corrected on the pattern layout and forgot to output the standard UI error messaging component elements.
4. Next Experiment:	Provide explicit examples of what good looks like to anchor both the logical design and UI requirements simultaneously.

Version 4: Layer Added — Examples of What Good Looks Like (Honest Failure Step)

The Prompt:

> *"Write a client-side settings form component for a Next.js App Router application that handles user name and phone updates using react-hook-form and zod validation. Phone validation must enforce E.164. Example of what good looks like for UI errors: {errors.phone && <p className=\"text-red-500 text-sm\">{errors.phone.message}</p>}"*

Output Excerpt:

> "```tsx
> // The AI generated a magnificent form UI but injected an older React 17 hook pattern (`useFormState`) that breaks modern Next.js layout tree architectures.
> export default function Form() {
> const [state, action] = useFormState(saveAction, null);
> ```"

1. What Changed:	Injected specific frontend UI code formatting reference templates for structural handling.
2. What Improved:	The template rendering became production-ready with dynamic Tailwind text styling alerts rendering correctly next to input zones.
3. What Failed:	CRITICAL FAILURE: The model mixed architectural metaphors, injecting an unrelated `useFormState` Hook that broke the application hierarchy layout.
4. Next Experiment:	Enforce rigorous verification requirements and explicit review instructions to eliminate illegal hook rendering patterns.

Version 5: Layer Added — Verification Requirements & Review Instructions

The Prompt:

"Write a client-side settings form component for a Next.js App Router application that handles user name and phone updates using react-hook-form and zod validation. Phone validation must enforce E.164. Follow this pattern for errors: {errors.phone && <p className=\"text-red-500 text-sm\">{errors.phone.message}</p>}. Verification: You must ensure that no server-action experimental form hooks are included. The component must rely strictly on standard react-hook-form submission hooks (`handleSubmit`). Double-check that all components are fully accessible using proper semantic HTML with ARIA properties."

Output Excerpt:

"```tsx
"use client";
import { useForm } from "react-hook-form";
// Fully isolated, valid hook logic with correct standard onSubmit triggers, complete semantic labels, and matching ARIA aria-invalid descriptors.
const onSubmit = data => console.log(data);
return (

    <label htmlFor="phone">Phone Number</label>
    <input id="phone" aria-invalid={errors.phone ? "true" : "false"} ... />
```"

| | |
|---|---|
| **1. What Changed:** | Added strict structural prohibitions against foreign API wrappers, along with explicit accessibility instructions. |
| **2. What Improved:** | The code purged the broken hooks completely, defaulted back to highly stable programmatic submit flows, and introduced robust semantic structure (proper HTML IDs, explicit `htmlFor` configurations, and functional dynamic `aria-invalid` bindings). |
| **3. What Failed:** | The file was completely error-free, but it lacked a real network latency loading layout element. |
| **4. Next Experiment:** | Consolidate all iterations into a production-ready, clean master prompt template for independent usage. |
```

Final Reusable Engineered Prompt

```
Act as an expert frontend engineer specializing in Next.js App Router
architectures.

Task: Generate a clean, standalone production-ready client-side form component
for updating a user's account settings profile (specifically updating Full Name
and International Phone Number).

Architectural Requirements:
1. Must be a Next.js App Router client component ("use client").
2. Form state management must be powered exclusively by 'react-hook-form'.
3. Input validations must be processed via a Zod verification schema passed
through a 'zodResolver'.

Constraint Rules:
- Full Name: String field, required, minimum 2 characters.
- Phone Number: String field, must strictly validate against an international
E.164 phone numbering standard format using a rigorous regex check (e.g., must
begin with a plus sign followed by country code and digits, no spaces).

UI & Accessibility Standards:
- Must use clean, semantic HTML layout items (<form>, <label>, <input>).
- All input elements must feature corresponding 'htmlFor' label connections and
dynamic 'aria-invalid' bindings linked to active error fields.
- Validation errors must render immediately below their respective inputs using
this specific styling structure:
  {errors.fieldName && <p className="text-red-500 text-sm mt-
1">{errors.fieldName.message}</p>}

Verification Checks:
Review your code before outputting. Ensure no older React experimental hooks or
conflicting state managers are bundled. Do not combine this with raw server
action bindings inside the form element—process everything securely through a
standard, isolated onSubmit handler function.
```